第七次讨论课
经典算法设计技术研讨



经典算法设计技术研讨

贪心法



1. 概述
2. 算法思想
3. 求解过程
4. 算法具体步骤
5. 性能分析

引言

假设存在下面需要付费的广播台，以及广播台信号可以覆盖的地区。

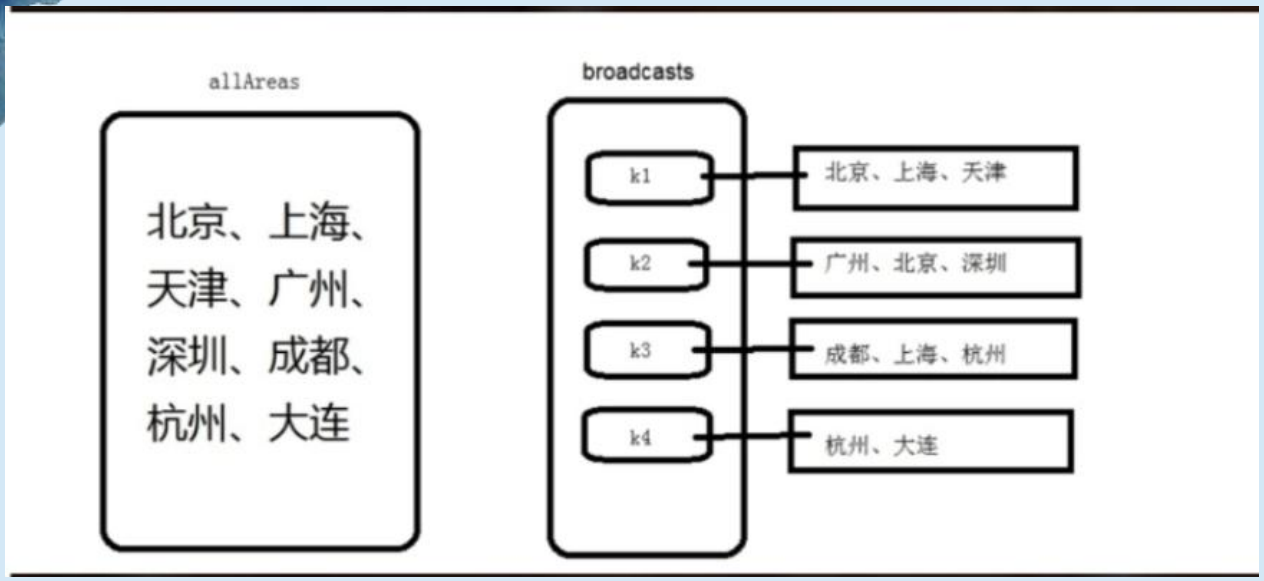
如何选择最少的广播台，让所有的地区都可以接收到信号

广播台	覆盖地区
K1	"北京","上海","天津"
K2	"广州","北京","深圳"
K3	"成都","上海","杭州"
K4	"上海","天津"
K5	"杭州","大连"

https://blog.csdn.net/weixin_45857153

那么这个问题我们使用什么方式解决呢？答：贪心算法

解决策略



1、遍历所有的广播电台，找到一个覆盖了最多未覆盖的地区的电台
2、将这个电台加入到一个集合中，把该电台覆盖的地区在下次比较时去掉
3、重复第1步直到覆盖了全部的地区

注意

贪婪算法所得到的结果不一定是最优的结果(有时候会是最优解)，但是都是相对近似(接近)最优解的结果K2, K3,K4,K5 也可以覆盖全部地区，如果K2 的使用成本低于K1,那么上题的 K1, K2, K3, K5 虽然是满足条件，但是并不是最优的

贪心算法 概述

贪心算法（又称贪婪算法）是指，在对问题求解时，总是做出在当前看来是最好的选择。也就是说，不从整体最优上加以考虑，算法得到的是在某种意义上的局部最优解 。

贪心算法不是对所有问题都能得到整体最优解，关键是贪心策略的选择 。

算法思想

贪心算法一般按如下步骤进行：

①建立数学模型来描述问题 。
②把求解的问题分成若干个子问题 。
③对每个子问题求解，得到子问题的局部最优解 。
④把子问题的解局部最优解合成原来解问题的一个解 。

贪心算法是一种对某些求最优解问题的更简单、更迅速的设计技术。贪心算法的特点是一步一步地进行，常以当前情况为基础根据某个优化测度作最优选择，而不考虑各种可能的整体情况，省去了为找最优解要穷尽所有可能而必须耗费的大量时间。

贪心算法采用自顶向下，以迭代的方法做出相继的贪心选择，每做一次贪心选择，就将所求问题简化为一个规模更小的子问题，通过每一步贪心选择，可得到问题的一个最优解。虽然每一步上都要保证能获得局部最优解，但由此产生的全局解有时不一定是最优的，所以贪心算法不要回溯 。

算法特性

贪心算法可解决的问题通常大部分都有如下的特性：

1、有一个以最优方式来解决的问题。为了构造问题的解决方案，有一个候选的对象的集合：比如不同面值的硬币。

2、随着算法的进行，将积累起其他两个集合：一个包含已经被考虑过并被选出的候选对象，另一个包含已经被考虑过但被丢弃的候选对象。

3、有一个函数来检查一个候选对象的集合是否提供了问题的解答。该函数不考虑此时的解决方法是否最优 。

4、还有一个函数检查是否一个候选对象的集合是可行的，即是否可能往该集合上添加更多的候选对象以获得一个解。和上一个函数一样，此时不考虑解决方法的最优性 。

5、选择函数可以指出哪一个剩余的候选对象最有希望构成问题的解 。

6、最后，目标函数给出解的值 。

使用条件

利用贪心法求解的问题应具备如下2个特征：

1、贪心选择性质
一个问题的整体最优解可通过一系列局部的最优解的选择达到，并且每次的选择可以依赖以前作出的选择，但不依赖于后面要作出的选择。这就是贪心选择性质。对于一个具体问题，要确定它是否具有贪心选择性质，必须证明每一步所作的贪心选择最终导致问题的整体最优解。

2、最优子结构性质
当一个问题的最优解包含其子问题的最优解时，称此问题具有最优子结构性质。问题的最优子结构性质是该问题可用贪心法求解的关键所在。在实际应用中，至于什么问题具有什么样的贪心选择性质是不确定的，需要具体问题具体分析 。

解题策略

贪心算法不从整体最优上加以考虑，所做出的仅是在某种意义上的局部最优选择。使用贪心策略要注意局部最优与全局最优的关系，选择当前的局部最优并不一定能推导出问题的全局最优。贪心策略解题需要解决以下两个问题：

1、该问题是否适合使用贪心策略求解，也就是该问题是否具有贪心选择性质；

2、制定贪心策略，以达到问题的最优解或较优解 。

要确定一个问题是否适合用贪心算法求解，必须证明每一步所作的贪心选择最终导致问题的整体最优解。证明的大致过程为：首先考察问题的一个整体最优解，并证明可修改这个最优解，使其以贪心选择开始，做了贪心选择后，原问题简化为规模更小的类似子问题。然后用数学归纳法证明通过每一步做贪心选择，最终可得到问题的整体最优解。

存在问题

贪心算法也存在如下问题：

1、不能保证解是最佳的。因为贪心算法总是从局部出发，并没从整体考虑；

2、贪心算法一般用来解决求最大或最小解；

3、贪心算法只能确定某些问题的可行性范围。

应用实例

例如，平时购物找零钱时，为使找回的零钱的硬币数最少，不要求找零钱的所有方案，而是从最大面值的币种开始，按递减的顺序考虑各面额，先尽量用大面值的面额，当不足大面值时才去考虑下一个较小面值，这就是贪心算法 。

有很多经典的应用，比如**霍夫曼编码**，**普利姆和克鲁斯卡尔**最小生成树算法，还有**迪杰斯特拉单源最短路径算法**，都是使用了这种思维。

例题

小船过河问题

1、题目：N个人过河，船每次只能坐两个人，船载每个人过河的所需时间不同t[i]，每次过河的时间为船上的人的较慢的那个，求最快的过河时间。(船划过去要有一个人划回来)

【Input】
第一行输入人数n;
第二行输入每个人过河所需的时间;
【Output】
输出需要的最少时间
【Sample Input】
4
1 2 5 10
【Sample Output】
17

两种方法，较容易想到的是第一种方式：
第一种办法：先让1 2过去（2分钟），1回来（1分钟），1 5过去（5分钟），1回来（1分钟），1 10再过去（10分钟），总共需要19分钟就可以让四个人都过去。
而正确答案是第二种办法：先让1 2过去（2分钟），1回来（1分钟），5 10过去（10分钟），2回来（2分钟），1 2再过去（2分钟），总共需要17分钟就可以让四个人都过去。

例题

本题的关键在于把最慢的和次慢的两个人运过河，因此只要大于四个人的情况下：
只需要将这两种方式这两个人运过河的时间做对比：
第一种：T4+T1+T3+T1
第二种：T2+T1+T4+T2
对比之后取短的就行

例题源码

```
int Cross(int a[],int n)
{
    sort(a,a+n);//这里运用了内部的排序 想提升时间可以运用快排 上面的代码是快排
    int time=0;
    while(n>3)//负责减最慢和次慢的人运过河
    {
        if( (2*a[0]+a[n-2]+a[n-1])<=(2*a[1]+a[0]+a[n-1]) )
            time=time+2*a[0]+a[n-2]+a[n-1];
        else
            time=time+2*a[1]+a[0]+a[n-1];
        n=n-2;//运过去两个人
    }
    if(n==3)
        time=time+a[0]+a[1]+a[n-1];
    if(n==2)
        time=time+a[n-1];
    if(n==1)
        time=time+a[0];
    return time;
}
```

```
int main()
{
    int n;//输入一共多少个人过河
    cin>>n;
    int a[255];
    memset(a,0,sizeof(a));//初始化
    for(int i=0;i<n;i++)
        cin>>a[i];//输入每个人过河的时间
    cout<<Cross(a,n)<<endl;
    return 1;
}
```

论文赏析

调度使用多架无人机的软件包交付的贪心算法

Greedy Algorithms for Scheduling Package Delivery with Multiple Drones

Francesco Betti Sorbelli
University of Perugia,
Italy
francesco.bettisorbelli@unipg.it

Federico Corò
Missouri University of Science and Technology, United States
federico.coro@mst.edu

Sajal K. Das
Missouri University of Science and Technology, United States
sdas@mst.edu

Lorenzo Palazzetti
University of Florence,
Italy
lorenzo.palazzetti@unifi.it

Cristina M. Pinotti
University of Perugia,
Italy
cristina.pinotti@unipg.it

论文赏析-摘要

ABSTRACT

Unmanned Aerial Vehicles (or drones) can be used for a myriad of civil applications, such as search and rescue, precision agriculture, or last-mile package delivery. Interestingly, the cooperation between drones and ground vehicles (trucks) can even enhance the quality of service. In this paper, we investigate the symbiosis among a truck and multiple drones in a last-mile package delivery scenario, introducing the Multiple Drone-Delivery Scheduling Problem (MDSP). From the main depot, a truck takes care of transporting a team of drones that will be used to deliver packages to customers. Each delivery is associated with a drone's energy cost, a reward that characterizes the priority of the delivery, and a time interval representing the launch and rendezvous times from and to the truck. The objective of MDSP is to find an optimal scheduling for the drones that maximizes the overall reward subject to the drone's battery capacity while ensuring that the same drone performs deliveries whose time intervals do not intersect. After showing that MDSP is an NP-hard problem, we devise an optimal Integer Linear Programming (ILP) formulation for it. Consequently, we design a heuristic algorithm for the single drone case and two more heuristic algorithms for the multiple drone case. Finally, we thoroughly compare the performance of our presented algorithms on different synthetic datasets.

在本文中，我们研究了卡车和多架无人机在最后一英里包裹交付场景中的共生关系，并介绍了多架无人机交付调度问题(MDSP)。

从主仓库，一辆卡车负责运送一组无人机，这些无人机将用于将包裹运送给客户。每一次交付都与无人机的**能源成本、交付优先级的奖励、从卡车和到卡车的发射和会合时间的时间间隔**有关。

//给出了三个影响因素

MDSP的目标是为无人机找到一个**最优的调度，最大化无人机的电池容量的总体奖励**，同时确保**同一无人机执行时间间隔不相交的交付**。

//给出了两个限制条件，求限制条件下的问题答案

在证明了MDSP是一个np困难问题之后，我们设计了一个最优的整数线性规划(ILP)公式。因此，我们设计了单一无人机的启发式算法和多无人机的启发式算法。

最后，我们彻底地比较了我们提出的算法在不同的合成数据集上的性能。

论文赏析-Introduction

第一部分Introduction,乍看没有讲什么特别相关的内容。

We make the following contributions in this paper.

- We propose an optimization problem, called Multiple Drone-Delivery Scheduling Problem (MDSP), and prove its *NP*-hardness, devising also the optimal Integer Linear Programming (ILP) formulation which is only suitable for small-sized instances.
- We design a heuristic algorithm for the single drone case, and two heuristic algorithms for the multiple drones case, which can be run for larger instances.
- We extensively compare the performance of our algorithms on synthetic datasets.

我们在本文中做以下贡献。
•提出了一个优化问题，即多重无人机递送调度问题(MDSP)，并证明了它的不稳定性，并设计了仅适用于小实例的最优整数线性规划(ILP)公式。**//优化问题并设计了线性规划公式**
•我们为单个无人机案例设计了一种启发式算法，为多个无人机案例设计了两种启发式算法，可以在更大的实例中运行。
•我们广泛地**比较了我们的算法在合成数据集上的性能**

还给出了文章的具体结构
本文的其余部分的结构如下。
第二节调查相关工作。
第3节介绍了显示其np硬度的MDSP，并提出了**最佳的ILP配方。**
第4节为单个和多个无人机的案例**设计了MDSP的算法。//这里应该是重点**
第5节评估了合成数据集上的算法。
最后，第6节给出了结论和未来的研究方向。

论文赏析-Related Work

2 RELATED WORK

This section reviews the literature related to the problem of delivering packages with the help of trucks and drones. We categorize the existing works based on whether the deliveries are done via both trucks and drones, or only by drones in which case the truck is only a carrier.

2.1 Delivery by Both Trucks and Drones

The truck-drone cooperation in a last-mile delivery scenario has been investigated for the first time when the flying sidekicks traveling salesman problem (FSTSP) has been introduced in [22]. The FSTSP is a particular case of the original TSP, where drones take off from the truck, fly to deliver packages to customers, and then go back to the truck in another place. Both the vehicles can perform deliveries, but each has to stop waiting for the other at the rendezvous location. The authors proposed an optimal mixed-integer linear programming formulation (MILP) and two heuristic solutions for solving the FSTSP. A similar approach is also studied by the same authors in [23] for multiple drones.

A greedy heuristic algorithm for the truck-and-drone delivery system is proposed in [8]. The heuristic starts with a solution for the truck only, which comprises nodes (i.e., locations) provided by a TSP instance. Then it greedily builds short sub-paths for the drones by excluding some nodes from the truck's path in order to reduce

propose to simultaneously employ trucks, truck-carried drones, and independent drones to construct a more efficient truck-drone parcel delivery system. A novel routing and scheduling algorithm are proposed to solve the hybrid parcel delivery problem.

2.2 Delivery by Drones Only

A similar work to ours is proposed in [4], where the authors consider a predefined route for the truck, and the problem is to optimize the planning of the drone's flights to/from the truck while serving the customers. Unlike us, however, their goal is to determine the launch and meeting points between drones and trucks. On the other hand, in our work these points are provided in input, and our goal is to determine a subset of deliveries to maximize the revenue. Furthermore, these points are calculated in such a way that the intervals generated do not intersect with each other, thus making all deliveries conflict-free. Importantly, in [4] the authors assume that drones do not have a battery constraint. While their goal is to reduce total makespan (i.e., the time difference between the start and end of a delivery sequence), our goal is to maximize the revenue/reward for deliveries.

The authors in [28] studied the cooperation between trucks and drones from a different point of view. In this work, the truck is only concerned with transporting the drones in order to improve the delivery system and make it more efficient. The objective is to determine a valid combination of the drone's characteristics (i.e.,

[1] Arnold, R., Mezzacappa, E., Jablonski, M., Abruzzo, B., and Jablonski, J. Experimentation for optimization of heterogeneous drone swarm configurations: terrain and distribution. In *Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications III* (2021), vol. 11746, International Society for Optics and Photonics, p. 1174625.

[2] Betti Sorbelli, F., Corò, F., Das, S. K., and Pinotti, C. M. Energy-constrained delivery of goods with drones under varying wind conditions. *IEEE Transactions on Intelligent Transportation Systems* (2020), 1–13.

[3] Betti Sorbelli, F., Pinotti, C. M., Silvestri, S., and Das, S. K. Measurement errors in range-based localization algorithms for uavs: Analysis and experimentation. *IEEE Transactions on Mobile Computing (TMC)* (2020).

[4] Boysen, N., Briskorn, D., Fedtke, S., and Schwerdfeger, S. Drone delivery from trucks: Drone scheduling for given truck routes. *Networks 72*, 4 (2018), 506–527.

第二部分是关联工作，引用了一些**前人已经做出的贡献**，包括一些之前针对卡车、无人机的运输问题做出的研究。作者还贴心地分了类。

论文赏析-PROBLEM FORMULATION

问题终于来了。

我们一行一行看。

设A 为代表我们最后交付一英里（应用程序）场景的二维**交付区域**。让$\psi \in A$ 作为交付开始的仓库。仓库的位置位于（$x\ \psi$，$y\ \psi$），假定是在（0,0）中的笛卡尔坐标系的原点。这样的交付区域还包括道路和客户位置。

在仓库，一辆卡车负责运输一队m无人机d 1，……，d m具有与A 相同的交付能力。卡车不执行交付；它只在A 内携带无人机。

设ρ j 是一条由两个端点λ j 和λ j +1分隔的**直线道路**，其中为j =0，…，r 和{λ 1，…，λ r }$\subset A$.卡车离开仓库ψ，沿着直线段ρ 0=ψ λ 1，访问第一个终点λ 1，然后沿着段ρ 1=λ 1λ 2，访问下一个终点λ 2，等等，直到最后一个终点λ r ，最终返回depotψ 。

这些片段形成一个**封闭的路径（循环）C**，由端点序列λ 0，λ 1，…，λ r ，λ r +1，使λ 0=λ r +1=ψ 。显然，r +1相邻的道路ρ 0，…，ρ r 定义了C 。

让D **={δ 1，……，δ n }**$\subset A$ 是**无人机服务的n （即客户）**。每个客户的（交付）点δ i 都有一对坐标（x δ i ，y δ i ）$\in A$ ，用于i =1，……，n 。图1说明了包含道路和客户的交付区域A 。

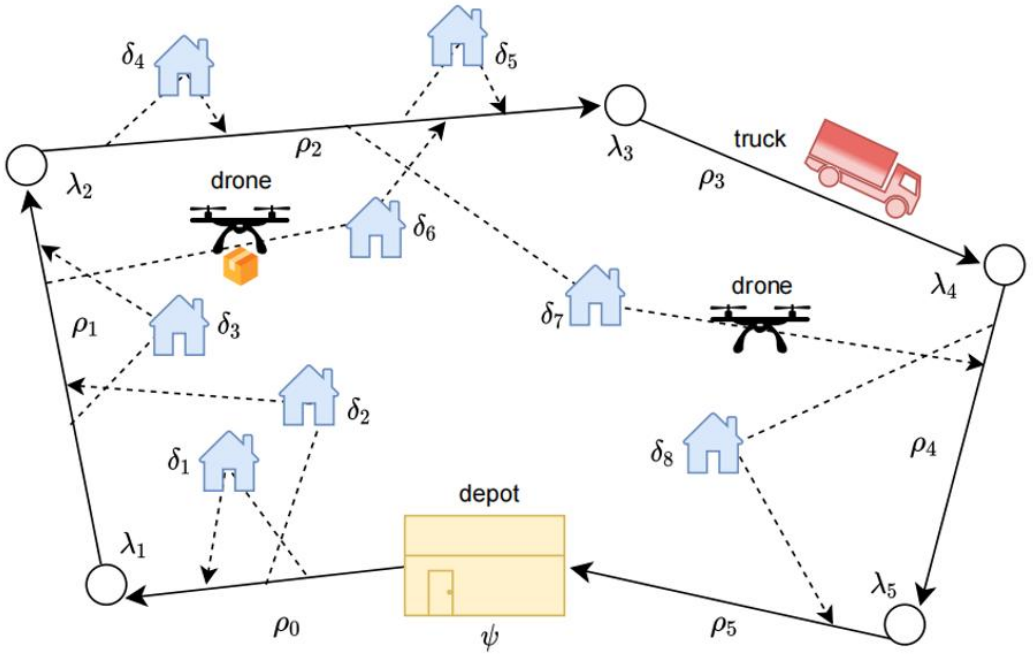


Figure 1: An example delivery area A with 6 roads ρ_j and 8 customers δ_i to serve. The depot ψ is at $(0, 0)$; The truck's path is the solid line, while the drones' paths are the dashed lines.

图1：一个例子交付区A ，有6条道路ρ j 和8个客户δ i 提供服务。ψ 仓库在（0,0）；卡车的路径是实线，而无人机的路径是虚线。

论文赏析-PROBLEM FORMULATION

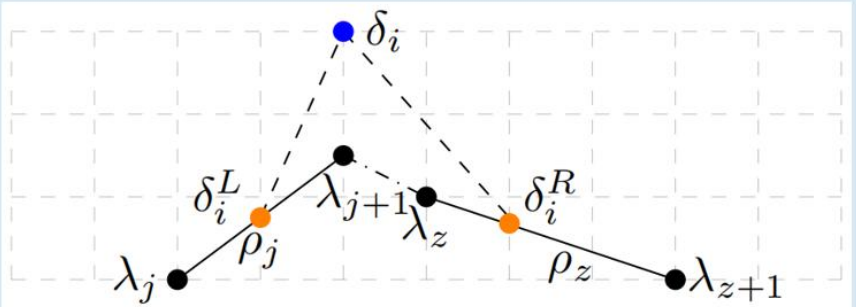


Figure 2: Two non-adjacent roads with a delivery to do: the launch and rendezvous points are highlighted.

让$w\ i\ \geq 0$为无人机的**能量成本（或重量）**，以执行一次交付$\delta\ i\ $从卡车飞行的能量。让$B\ \geq 0$成为无人机的电池容量方面的能源预算，这限制了它可能所能完成的交付数量。请注意，如果无人机目前的剩余能量不足以进行额外的飞行，那么就不可能进行进一步的交付，除非更换一个新的电池。然而，我们假设无人机只能依赖于最初的单次电池充电。

让$p\ i\ \geq 0$作为**执行交付$\delta\ i\ $的奖励**。奖励的意义是指**高级用户比普通用户拥有更高的优先级**。例如，快递公司根据以下规则提供不同的订阅服务：你支付的钱越多，你收到包裹的速度就越快。在我们的背景中，交付$\delta\ i\ $的优先级越高，奖励价值$p\ i\ $越大。因此，为了满足优质用户，无人机的调度需要优先考虑交付，首先保证属于。

让$t\ \geq 0$，表示**时间**。让$t\ 0=0$作为卡车离开$\lambda\ 0=\psi$位置的仓库时的初始时间，以便在A区域内进行交付。同样，让$t\ r+1$是$\lambda\ r\ +1=\psi$交付应用程序的最后一次时间。重要的是要观察卡车沿着预定义的终点序列行驶路线，在一个特定的方向上精确访问一次。因此，如果卡车计划通过终点$\lambda\ i\ $和$\lambda\ j\ $，那么使用$0\leq i$

论文赏析-PROBLEM FORMULATION

问题1（多架无人机交付调度问题(MDSP)）。让δ 1，…，δ n 是**交付位置**，m是**无人机数量**，B 无人机的**电池预算**。MDSP的目的是为1≤p =q ≤m，找到一个具有S ∗ p ∩S ∗ q =∅ 的m可行子集的家族S ∗ ={S ∗ 1，…，S ∗ m}⊆I ，从而使总体奖励P（S ）最大化。

算法为：

S ∗ = arg max P (S)

S ={S 1,...,S m }⊆I

such that C (S i) ≤ B ∀i = 1, . . . ,m

论文赏析-PROBLEM FORMULATION

定义交付和约束集合的另一种方法是观察**intervals**I 的集合可以被描述为一个区间图，这是弦图的一种特殊情况。在MDSP中，交付集合D 确定可以与开始和结束时间成对**兼容或冲突**的间隔I 集。给定MDSP的一个实例，我们可以沿着**时间线**可视化时间间隔及其兼容性。图3显示了与图1对应的时间间隔，其中I 2与I 1和I 3都有冲突，而I 1和I 3则是兼容的。

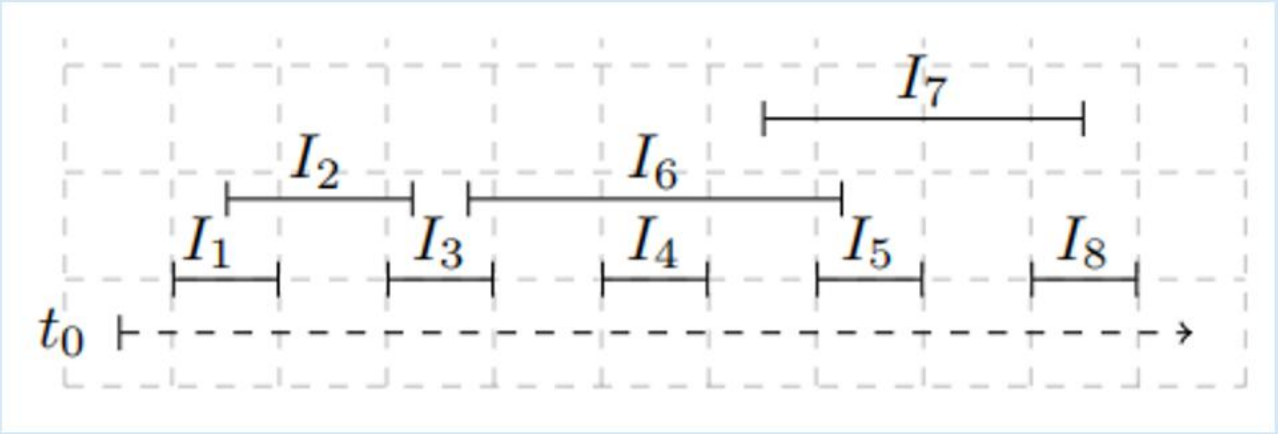


图3：与图1对应的交货间隔时间

论文赏析-PROBLEM FORMULATION

计算两个区间是否冲突的一个简单方法是考虑**区间图**表示。形式上，在无向区间图，$G=(V,E)$，中，顶点集$V=I$，，其中每个顶点唯一对应一个区间$I\ i\ \in I$，for$i$ =1，...，n；和一个边（$I\ i$，$I\ j$）$\in E$，表示**传递$\delta\ i$ 和$\delta\ j$ 不兼容**，这意味着$I\ i\ \cap I\ j\ =\varnothing$.例如，在图3中，$I$ 1和I 3可以在没有冲突的情况下执行，即（I 1，I 3）$\in E$；而I 2同时与I 1和I 3不兼容，这意味着（I 2，I 1），（I 2，I 3）$\in E$。请注意，顶点I 6有四个冲突，因为它的度是4。

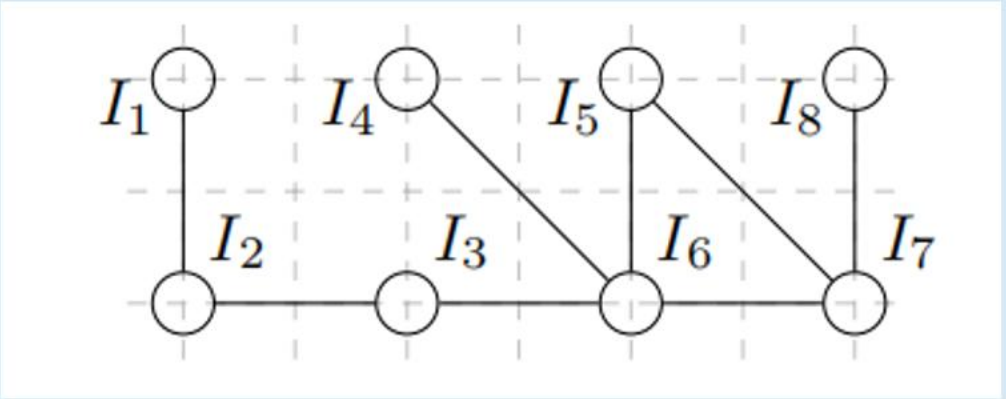


图4：图1中的示例的间隔图G

论文赏析-PROBLEM FORMULATION

证明

下面，我们通过证明经典的0-1背包问题(KP)是当间隔**无冲突**时MDSP的一种**特殊情况**，证明了MDSP是一个np困难问题。定理1

MDSP是np-困难的。我们的方法是通过减少KP，这是已知的np难[19]，定义如下。给定一组X ={1，…，n }，每个项目都与成本$w\ i$ 和奖励$p\ i$ 相关，以及一个背包容量c ，KP是找到X 的一个子集，**最大化奖励的总和**，满足容量约束。给定KP的一个实例，我们将其转换为MDSP的一个实例如下：我们首先将MDSP的能量预算设置为等于背包的容量约束，即B =c 。然后，我们从一组项目X 开始创建一组交付D ，如下：对于每个项目$x\ i$ ∈X ，创建一个具有相同成本和奖励的交付$\delta\ i$ ∈D 。因此，我们可以观察到，在相应的MDSP实例中，交付是**成对兼容**的。因此，KP的解是MDSP的解，反之亦然。这个约简需要多项式的时间。因此，证明。

In the following we demonstrate that MDSP is an *NP*-hard problem by showing that the classic 0–1 Knapsack Problem (KP) is a special case of MDSP when intervals are conflict-free.

THEOREM 1. *MDSP is NP-hard.*

PROOF (SKETCH). Our approach is by reduction from KP, which is known to be *NP*-hard [19] and defined as follows.

Given a set $X = \{1, \ldots, n\}$ of n items, each associated with a cost w_i and reward p_i, and a knapsack of capacity c, the KP is to find a subset of X that maximizes the sum of the rewards, satisfying the capacity constraint. Given an instance of KP, we translate it as an instance of MDSP as follows: we first set the energy budget of MDSP equal to the capacity constraint of the knapsack, i.e., $B = c$. Then we create a set of deliveries D starting from the set of items X as follows: for each item $x_i \in X$, create a delivery $\delta_i \in D$ with equal cost and reward. Hence, we can observe that the deliveries are pairwise compatible in the corresponding MDSP instance. Thus, a solution for KP is a solution for MDSP and vice versa. This reduction takes polynomial time. Hence, proven. □

论文赏析-PROBLEM FORMULATION

为了在有m架无人机的一般情况下最优地求解MDSP，我们可以使用**ILP公式**。我们列举了交付为N={1，……，n }，和无人机为M={1，……，m}。设$x\ i\ j\ \in${0,1}为决策变量，如果由无人机$i\ \in$M完成交付$j\ \in$N，则为**1**；否则为**0**。因此，ILP公式是由一个目标函数和一些约束形成的：

$$\max \sum_{i=1}^{m}\sum_{j=1}^{n} p_j x_{ij} \tag{1}$$

subject to:

$$\sum_{j=1}^{n} w_j x_{ij} \le B, \qquad \forall i \in \mathcal{M} \tag{2}$$

$$\sum_{i=1}^{m} x_{ij} \le 1, \qquad \forall j \in \mathcal{N} \tag{3}$$

$$x_{ij} + x_{ik} \le 1, \qquad \forall i \in \mathcal{M}; \forall j,k \in \mathcal{N} s.t.\ I_j \cap I_k \neq \varnothing \tag{4}$$

$$x_{ij} \in \{0,1\}, \qquad \forall i \in \mathcal{M}, \forall j \in \mathcal{N} \tag{5}$$

表达式（1）中的目标函数使用m无人机舰队**最大化整体奖励**。约束（2）规定，每架无人机在电池方面都有一个**能源预算**B；约束（3）强制每次交付由不超过一个无人机执行；（4）确保只有在没有冲突时，同一架无人机才能执行交付。由于上述求解MDSP的**ILP公式**只适用于输入中的小实例

论文赏析-PROBLEM FORMULATION

这里提出了一种**单无人机MDSP启发式算法**，即最大比例单无人机(Mr-S)，以及两种**多无人机MDSP启发式算法**，即MaxCliqe多无人机(Mc-M)和最大比例多无人机(Mr-M)

第一种方法

Mr-S是一种用单一无人机解决MDSP的启发式方法，需要**O(n log$n$)**时间和**O（n ）**空间。在算法1中给出了Mr-S的伪代码。

Algorithm 1: The MR-S Algorithm

1 sort(I) s.t. $\frac{p_1}{w_1} \geq \ldots \geq \frac{p_n}{w_n}$
2 $SOL \leftarrow \emptyset$
3 **for** $i \in 1, \ldots, n$ **do**
4 **if** is-augmentable(I_i, SOL) **then**
5 $SOL \leftarrow SOL \cup I_i$
6 **return** SOL

在Mr-S中，我们首先根据奖励与能量成本的比率$p\ i\ w\ i$ 按**非递增顺序**对间隔进行排序（第1行，算法1）。然后，我们在当前的解决方案$S\ O\ L$ 中添加最佳可能的间隔$I\ i$ （即，具有最大的比率），它不会与已经选择的间隔产生冲突，并评估剩余的能量预算是否足够。这是通过可扩充的程序（第4行）来完成的。

论文赏析-PROBLEM FORMULATION

这里提出了一种单**无人机MDSP启发式算法**，即最大比例单无人机(Mr-S)，以及两种**多无人机MDSP启发式算法**，即MaxCliqe多无人机(Mc-M)和最大比例多无人机(Mr-M)

第二种方法

启发式Mc-M在**O(**$m(n\ \log n + h\ (n))$时间和**O**(n)空间中使用多个无人机解决MDSP，其中$h\ (n)$是其中使用的子例程所需的时间。在算法2中给出了Mc-M的伪代码。

Algorithm 2: The Mc-M Algorithm

1 $\hat{I} \leftarrow I, m' \leftarrow m, SOL \leftarrow \emptyset$
2 **while** $\hat{I} \neq \emptyset$ **do**
3 $\quad \{S_1, \ldots, S_\omega\} \leftarrow$ create-subsets$(\hat{I})$
4 $\quad$ sort(S_i) s.t. $\mathcal{P}(S_i) \geq \mathcal{P}(S_{i+1})$
5 $\quad SOL \leftarrow SOL \cup \{S_1, \ldots, S_{\min\{\omega, m'\}}\}$
6 $\quad \hat{I} \leftarrow \hat{I} \setminus \bigcup S_i\}$
7 $\quad m' \leftarrow m' - \min\{\omega, m'\}$
8 **return** SOL

在Mc-M中，我们根据**最大圈**ω的大小，根据剩余区间的大小对当前图进行划分。然后，我们为每个子分区分配一个无人机。这是通过创建子集过程（第3行，算法2）来完成的。最后，我们找到了一个包含先前计算出的解的全局解。

论文赏析-PROBLEM FORMULATION

这里提出了一种单**无人机MDSP启发式算法**，即最大比例单无人机(Mr-S)，以及两种**多无人机MDSP启发式算法**，即MaxCliqe多无人机(Mc-M)和最大比例多无人机(Mr-M)

第三种方法

Mr-M是一种用多架无人机求解MDSP的启发式算法，需要**O(m(n log$n$))**时间和**O（n ）**空间。在算法3中给出了Mr-M的伪代码。

Algorithm 3: The MR-M Algorithm

1 $\hat{I} \leftarrow I$
2 **for** $i \in 1, \ldots, m$ **do**
3 $\quad S_i \leftarrow$ MR-S($\hat{I}$)
4 $\quad \hat{I} \leftarrow \hat{I} \setminus S_i$
5 **return** $SOL \leftarrow \{S_1, \ldots, S_m\}$

最初，描述当前剩余间隔的集合I ^that用所有的交付I进行初始化（第1行，算法3）。在那之后，我们**迭代**（第2行）在I ^（第3行）上的Mr-S的m次（因为无人机舰队），其中包含了尚未分配给无人机的时间间隔。在这个环路的末端，有针对无人机$d\ i$ 的中间解决方案$S\ i$ 。因此，剩余无人机的间隔减少了（第4行）。最终，我们将$S\ O\ L$ 作为一个全局解决方案返回，其中包含在前面的步骤（第5行）中计算出的子解的并集。

论文赏析-PROBLEM FORMULATION

这里**比较**了三种提出的启发式方法用单一无人机和多个无人机案例解决MDSP的性能，即获得的奖励。

关于分配给每个交付的奖励，我们根据Zipf分布[34]在[0,0.4,0.8,0,1.0]中改变θ 参数，**随机生成**它们作为[1,100]之间的整数。当θ =为0时，奖励在[1,100]中均匀分布；当θ ≥为0.8时，有一些大概率的奖励，许多奖励出现几次。能量成本和跨度时间是根据均匀分布均匀产生的，假设**四种跨度/能量配置Σi** （从低变异性的Σ1到高变异性的Σ4）。具体来说，在配置Σ1中，最大能量成本为2500kJ，最大跨度时间为1500s，在Σ2中，这对能量成本分别为5000kJ和10000s，以此类推。请注意，Σ1和Σ2总是允许可行的交付，而其他两个则不允许。此外，Σ4允许很长的跨度时间与卡车的旅行时间。最后，一旦生成跨度长度τ i ，则在t 0和t r +1−τ i 之间均匀生成发射时间t i L 。

About the rewards assigned to each delivery, we randomly generate them as an integer number between [1, 100] according to the Zipf distribution [34] varying the θ parameter in $[0, 0.4, 0.8, 1.0]$. When $\theta = 0$, the rewards are uniformly distributed in [1, 100]; when $\theta \geq 0.8$, there are a few rewards with a large probability and many rewards occurring a few times. The energy costs and the span times are uniformly generated according to the Uniform distribution, assuming four *span/energy configurations* Σ_i (from Σ_1 with low variability to Σ_4 with high variability). Specifically, in configuration Σ_1 the maximum energy cost is 2500 kJ and the maximum span time is 1500 s, in Σ_2 this pair is 5000 kJ and 10000 s, and so on. Note that, Σ_1 and Σ_2 always admit feasible deliveries, while the other two do not. Moreover, Σ_4 allows very long span times as large as the duration of the truck's tour. Finally, once the span length τ_i is generated, the launch time t_i^L is uniformly generated between t_0 and $t_{r+1} - \tau_i$.

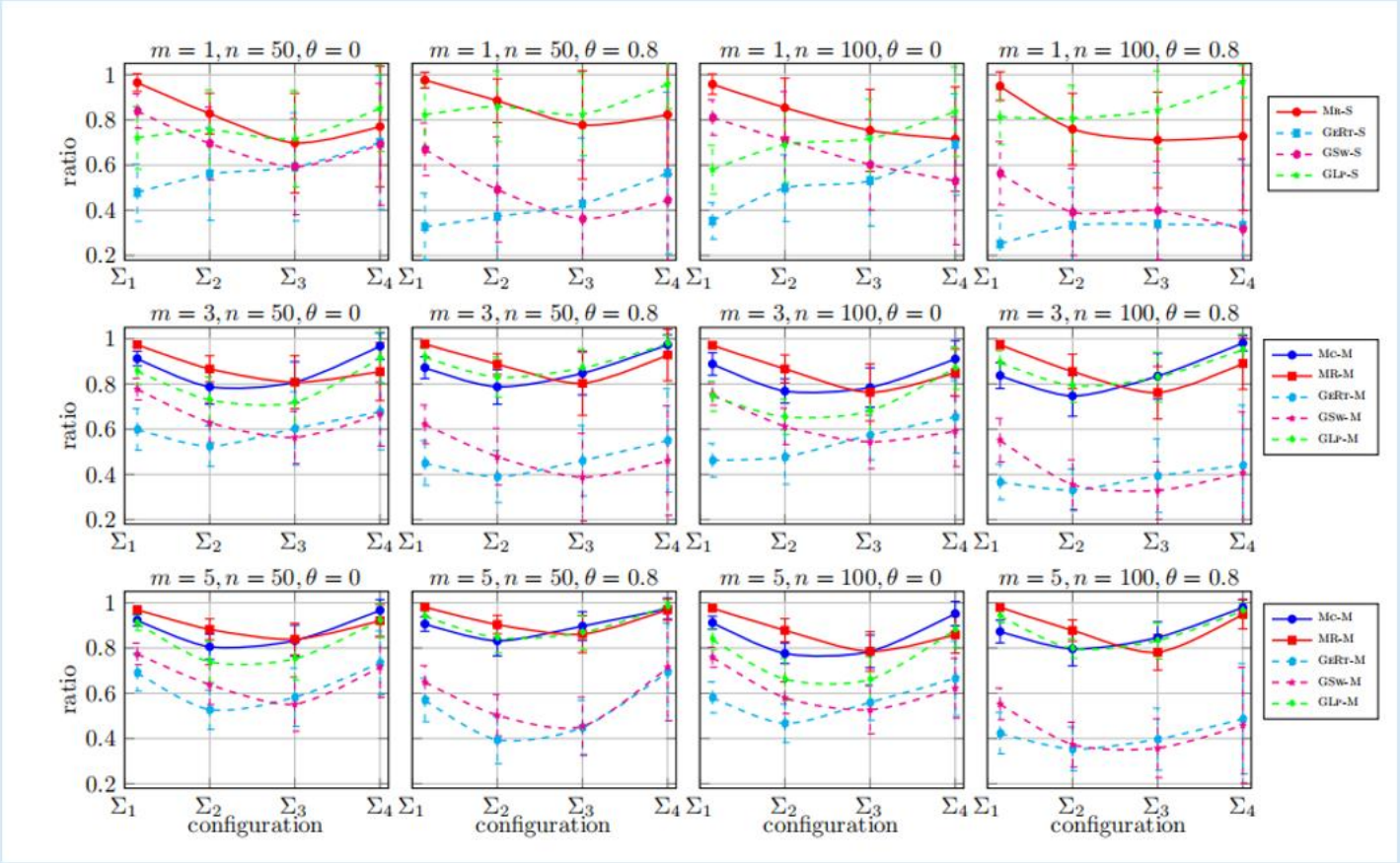
Table 2: Used input parameters.

Par.	Description	Value	Unit
d_s	drone's speed	20	m/s
d_p	drone's payload	5	kg
B	drone's budget	5000	kJ
n	number of deliveries	[25, 50, 75, 100]	–
m	number of drones	[1, 3, 5]	–
t_{r+1}	final time	30000	s
p	max reward	100	–
w	max energy cost	2500, 5000, 7500, 30000	kJ
τ	max span time	1500, 10000, 20000, 30000	s
θ	Zipf parameter	[0, 0.4, 0.8, 1.0]	–

论文赏析-PROBLEM FORMULATION

实验结果

图6说明了我们的算法在合成数据上的性能在收集奖励方面的性能。具体来说，在第一行中，我们展示了使用m =1（单个无人机）的MDSP的结果，而在另外两行中，我们分别展示了使用m =3和m =5无人机的MDSP的结果。x 轴报告四种不同的配置跨度/能量Σi ，而y 轴显示任何算法报告的奖励与最优算法报告的奖励之间的比率。显然，这个比率**小于或等于1**。



单一无人机场景。当m =1时，性能最好的算法是**Mr-S和GLp**，这两种算法在交付选择规则中都考虑了奖励。相反，GSw和GeRt的选择规则与奖励无关，它们表现出最差的表现。Zipf参数θ **=0.8**特别强调了这一趋势。在这种情况下，有一些大价值奖励和许多小价值奖励。因此，GLp有利于他们的选择。当奖励是均匀分配的（θ =0），Mr-S保证了比GLp更好的表现，可能是因为它在选择时也考虑成本（权重），而不是盲目地考虑奖励。相反，在存在不平衡奖励（θ **=0.8**）的情况下，**GLp**展示了最佳的解决方案，因为该算法可以选择最可回复的时间间隔，而不考虑任何其他因素。当θ =为0.8时，GLp的性能跳跃率很高。

论文赏析-PROBLEM FORMULATION

结论

本文研究了多架无人机交付调度问题(MDSP)，研究了在最后一英里的包裹交付场景中，卡车和多架无人机之间的合作。在证明了MDSP是一个np困难问题之后，我们提出了一个适用于输入中的小实例的最优ILP公式。然后，对于较大的实例，我们为单个和多个无人机提供了三种高效的启发式算法。最后，我们在合成数据集上评估了所提算法的性能。作为未来的工作，值得设计其他有效的算法，并寻找有保证的近似界。此外，研究多段多卡车场景，或允许无人机同时执行多次交付或在充电站充电，将是很有趣的。我们还计划研究一个更现实的动态环境，处理仓库和地面航空车辆之间的网络通信，解决可能的延误和新的/取消的交付。为了应对这些挑战，我们计划制定在线策略，重新安排送货时间。

参考资料

知乎：https://zhuanlan.zhihu.com/p/102182602；
CSDN：
https://blog.csdn.net/m0_46201544/article/details/122722787?ops_request_misc=&request_id=&biz_id=102&utm_term=%E4%B8%AD%E4%BD%8D%E6%95%B0%E5%88%87%E5%88%86&utm_medium=distribute.pc_search_result.none-task-blog-2~all~sobaiduweb~default-0-122722787.142^v9^control,157^v4^control&spm=1018.2226.3001.4187

THANKS!

HNU

2022

